CS3210 Parallel Computing

AY 25/26 Sem 2 — github/omgeta

1. Introduction

Parallel Computing is the simultaneous use of multiple processing units to solve problems efficiently.

Program Parallelization Steps:

i. Decomposition: split problem into parallel tasks
   - Granularity: size of task

ii. Scheduling: assign tasks to processes/threads
   - Orchestration: imposed synchronisation and communication of tasks to satisfy dependencies

iii. Mapping: bind processes/threads to hardware processing units (e.g. CPU cores)

Processes

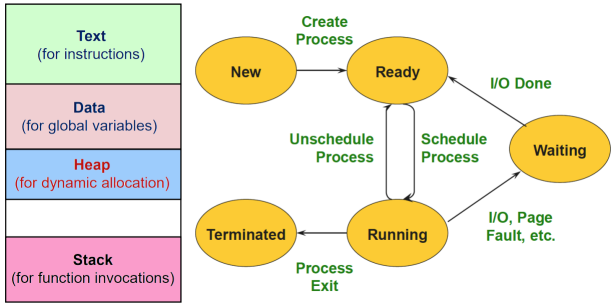
Process is an abstraction for a running program:

i. Process ID (PID): uniquely identifies a process

ii. Process State: indicates execution status

iii. Process Control Block: stores execution context (registers, resources, private address space)

– Costly: syscall overhead, data structures must be allocated, communication goes through OS



Interprocess Communication (IPC):

i. Shared Memory (e.g. locks, semaphores)

ii. Message Passing

iii. UNIX Pipes and Signals

Threads

Threads are independent execution flows within a process:

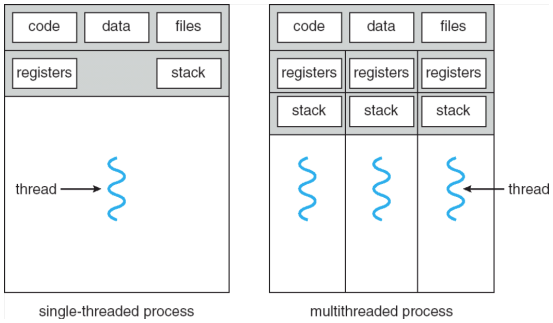
i. Shared: Resources, Address Space

ii. Private: Thread ID, Registers, "Stack" (diff. SP)

\+ Efficient: cheaper creation and context switching

\+ Resource Sharing: process resources can be shared

\+ Concurrent: multithreading on different cores



Implementations:

i. User-Threads are managed by a userspace library:
   + Fast: context switches have low overhead
   – No Parallelism: OS cannot map threads to different execution units
   – Threads blocking will block whole process

ii. Kernel-Threads are implemented in the OS:
   + Parallel: threads can run across many CPUs
   – Slower: thread operations are system calls

Mapping Models:

i. Many-to-One:
   - All user-level threads are mapped to one process
   - User library schedules user-threads

ii. One-to-One:
   - Each user-thread mapped to one kernel-thread
   - No user library scheduler needed

iii. Many-to-Many:
   - User-threads mapped to set of kernel-threads
   - Library scheduler may move user-threads to different kernel-threads

Synchronisation

Synchronisation restricts interleavings of concurrent threads to coordinate access to shared resources.

Critical Section protects resources from concurrent access:

i. Safety: nothing bad happens
   - Mutual Exclusion: at most 1 thread in CS

ii. Liveness: something good happens
   - Progress: if no thread is in CS, a waiting thread should be granted access
   - Bounded Wait: a waiting thread requesting to enter the CS, will eventually enter

iii. Performance: overhead is small w.r.t work done

Symptoms of Incorrect Synchronization:

i. Deadlock: all threads waiting; iff all conditions met
   - Mutual Exclusion: ≥ 1 resource held exclusively
   - Hold & Wait: ≥ 1 process holding one resource while waiting for another
   - No Pre-emption: resources must be yielded
   - Circular Wait: set of processes waiting in circles

ii. Starvation: a thread is unable to make progress
   - Livelock: due to deadlock avoidance

iii. Race Condition: bug from program outcome depending on unpredictable access order

iv. Data Race: two threads access a shared resource with no synchronisation, and one is a write

Mechanisms:

i. Lock: primitive `acquire()`, `release()`
   - Spinlock (busy-wait) vs. Mutex (blocking)
   - Uses hardware atomics (`TSL`) or toggle interrupts

ii. Semaphore: atomic counter ≥ 0; `wait()`, `signal()`
   - Binary/Mutex: represents single access
   - Counting/General: number of threads in CS $\leq N$
   - Can be manipulated by different thread
   - No relation to data being controlled

iii. Monitor: high-level implicit sync data structure

iv. Messages

Producer-Consumer

Processes share a buffer, where producers add items and consumers remove items when not empty.

Unbounded Buffer (`mutex = Sem(1)`, `items = Sem(0)`)

```
// Producer
x = produce();
mutex.wait();
  buffer.add(x);
mutex.signal();
items.signal();
```

```
// Consumer
items.wait();
mutex.wait();
  x = buffer.get();
mutex.signal();
consume(x);
```

Finite Buffer (`spaces = Sem(K)`):

```
// Producer
x = produce();
spaces.wait();
mutex.wait();
  buffer.add(x);
mutex.signal();
items.signal();
```

```
// Consumer
items.wait();
mutex.wait();
  x = buffer.get();
mutex.signal();
spaces.signal();
consume(x);
```

Lightswitch

```
class Lightswitch:
  def __init__(self):
    self.counter = 0
    self.mutex = Sem(1)

  def lock(self, sem):
    self.mutex.wait()
      self.counter++
      if self.counter == 1:
        sem.wait()
    self.mutex.signal()

  def unlock(self, sem):
    self.mutex.wait()
      self.counter--
      if self.counter == 0:
        sem.signal()
    self.mutex.signal()
```

Barrier

```
bool last = false;
mutex.wait();
count++;
if (count == N) {
  last = true;
  barrier.signal();
}
mutex.signal();

barrier.wait();
if (!last)
  barrier.signal();
```

Reader-Writer

Processes share a critical region, where readers can read simultaneously but writers must have exclusive access.

```
// Writer
empty.wait();
  write();
empty.signal();
```

```
// Reader
mutex.wait();
  readers++;
  if (readers == 1)
    empty.wait();
mutex.signal()
read();
mutex.wait();
  readers--;
  if (readers == 0)
    empty.signal();
mutex.signal()
```

Lightswitch:

```
// Writer
empty.wait();
  write();
empty.signal();
```

```
// Reader
rswitch.lock(empty);
  read();
rswitch.unlock(empty);
```

No Starvation (`gate = Sem(1)`):

```
// Writer
gate.wait();
empty.wait();
  write();
gate.signal();
empty.signal();
```

```
// Reader
gate.wait();
gate.signal();
rswitch.lock(empty);
  read();
rswitch.unlock(empty);
```

Priorities:

```
// Writer
wswitch.lock(noRead);
noWrite.wait();
  write();
noWrite.signal();
wswitch.unlock(noRead);
```

```
// Reader
noRead.wait();
rswitch.lock(noWrite);
noRead.signal();
  read();
rswitch.unlock(noWrite);
```

Dining Philosophers

N philosophers around a circular table, with a single chopstick between. 2 chopsticks are needed to eat.

```
// Philosopher
void p(int i) {
  while (1) {
    // think
    take(i);
    eat();
    put(i);
  }
}

void safe_to_eat
    (int i) {
  if (state[i] ==
   HUNGRY &&
   state[LEFT] !=
   EATING &&
   state[RIGHT] !=
   EATING) {
    state[i] =
   EATING;
    signal(s[i]);
  }
}
```

```
// Take Chopstick
void take(int i) {
  wait(mutex);
  state[i] =
   HUNGRY;
  safe_to_eat(i);
  signal(mutex);
  wait(s[i]);
}

// Put Chopstick
void put(int i) {
  wait(mutex);
  state[i] =
   THINKING;

   safe_to_eat(LEFT);

   safe_to_eat(RIGHT)
  signal(mutex);
}
```

Limited Eater Solution, when at most $N-1$ philosophers eat concurrently, it's guaranteed at least one can eat:

```
void philosopher(int i) {
  while (1) {
    // think
    wait(seats);
    wait(chopstick[LEFT]);
    wait(chopstick[RIGHT]);
    eat();
    signal(chopstick[RIGHT]);
    signal(chopstick[LEFT]);
    signal(seats);
  }
}
```

2. Architecture

Levels of Parallelism:

i. Bit-Level: operate on wider words
   - SIMD: operate on superword/vector registers

ii. Instruction-Level:
   - Pipeline: split execution into stages; multiple instructions can occupy different stages in same cycle (if no dependency hazards)
   - Superscalar: duplicate pipelines; issue multiple independent instructions per cycle

iii. Thread-Level: execute multiple threads on the same core in parallel (i.e. SMT)

iv. Processor-Level: execute multiple threads/processes in parallel on multiple cores/processors

Flynn Taxonomy of Parallel Architectures:

i. Single Instruction Single Data (SISD): single stream, each instruction on single data

ii. Single Instruction Multiple Data (SIMD): single stream, each instruction on multiple data

iii. Multiple Instruction Single Data (MISD): multiple streams, each instruction on same data

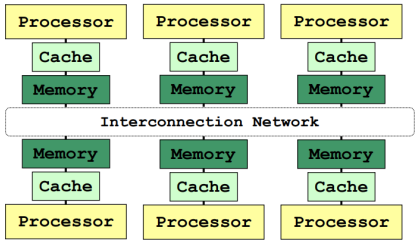
iv. Multiple Instruction Multiple Data (MIMD): per-PU stream to work on its own data

Distributed-Memory

Distributed-Memory Systems are multicomputer systems:

i. Node: indep. unit of processor and private memory

ii. Interconnection Network: supports message passing for data exchange between nodes



Shared-Memory

Shared-Memory Systems are multiprocessor systems:

i. Shared Memory Provider: maintains shared address space abstraction across PUs

\+ No need to partition code or data

\+ Efficient communication: no need to physically move data between processors

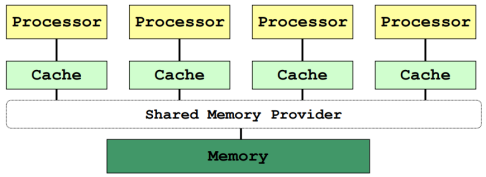
– Special synchonisation constructs required

– Limited scalability due to contention

Uniform Memory Access (UMA):

i. Same memory access latency for every PU

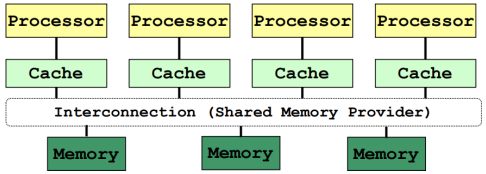
\+ Suitable with few processors due to contention



Non-Uniform Memory Access (NUMA):

i. Different memory access latency for every PU (e.g. local memory faster than remote)

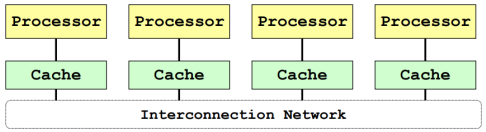
ii. Cache Coherent NUMA (ccNUMA): each node has coherent cache to reduce contention



Cache Only Memory Access (COMA):

i. Memory blocks behave as caches

ii. Data migrates dynamically and continuously according to cache coherence scheme



Cache Coherence

Caches reduce memory access latency and provide high bandwidth data transfer to CPU:

i. Cache Size: larger cache reduces misses but increases access time (addressing complexity)

ii. Block Size: unit of transfer; larger blocks improve spatial locality, but longer transfer time

Write Policy:

i. Write-through: write to both cache and memory
   \+ Always get newest value of memory block
   – Slower due to many memory accesses; mitigate with write buffer

ii. Write-back: write to cache only and set dirty bit; write to memory on cache block eviction
   \+ Fewer write operations to memory
   – Memory may contain invalid entries

Coherence defines behaviour of reads/writes to the same memory location. Properties:

i. Program Order: if P writes to X, n.f.w to X, P should read same value from X

ii. Write Propagation: if P_1 writes to X, n.f.w to X, P_2 should read same value from X eventually

iii. Transaction Serialization: if $V_1 \to X$, then $V_2 \to X$, all P_i should never read V_2 then V_1 from X

Implementations:

i. Software-based: use page-fault to propagate writes

ii. Snooping-based: cache controllers snoop/broadcast on shared bus

iii. Directory-based: central directory records sharing status; no broadcast; common in NUMA

Implications:

i. "Increase" memory latency and lower cache hit rate

ii. Cache Ping-Pong: 2 PUs write to same cache line; invalidations slow fetch from memory

iii. False Sharing: PUs write to different addresses on same cache line; avoid with padding

Memory Consistency

Memory Consistency constrains the order in which memory operations by a thread become visible (commit) to other threads for different memory locations.

Memory Ordering Requirements:

i. $W \to R$: write to X commits before read from Y
ii. $R \to R$: read from X commits before read from Y
iii. $R \to W$: read from X commits before write to Y
iv. $W \to W$: write to X commits before write to Y

Sequential Consistency Model (SC):

i. Preserves all four ordering requirements
ii. PUs issue memory operations in program order
iii. Interleaving of PUs' program orders is acceptable
– Can result in loss of performance

Relaxed Consistency Model:

i. Relax ordering only if no data dependencies between operations to same memory location:
   - $R \to W$: anti-dependence (WAR)
   - $W \to W$: output dependence (WAW)
   - $W \to R$: flow dependence (RAW)
ii. Override constructs can prevent certain re-orderings
iii. Total Store Ordering (TSO):
   - Relax $W \to R$: a PU may read B before its earlier write to A is visible to *all* other PUs
   - Write Atomicity: writes become visible to all PUs at the same time
iv. Processor Consistency (PC):
   - Relax $W \to R$
   - No Write Atomicity: a write may be seen by some PUs before others
v. Partial Store Ordering (PSO):
   - Relax $W \to R$
   - Write Atomicity
   - Relax $W \to W$: a later write to B can bypass an earlier write to A

\+ Hide latencies by overlapping indep. operations

3. Programming Models

Types of Parallelism:

i. Data Parallelism: split data among PUs (loop parallelism; SIMD/SPMD)
ii. Task Parallelism: split indep. tasks among PUs (statements, loops, function calls)
   - Task Dependency Graph: DAG representing control dependency between tasks
     – Critical Path Length: slowest completion
     – Deg. Concurrency: $\dfrac{\text{Total Work}}{\text{Critical Path Length}}$

Representation of Parallelism:

i. Implicit Parallelism: Haskell
ii. Implicit Scheduling: OpenMP
iii. Explicit Communication & Sync: MPI, Pthreads

Models of Coordination:

i. Shared Address Space: very little structure
   - All threads can access all shared variables; mutual exclusion via locks
   - Match shared-memory systems (UMA/NUMA); requires HW support; costly to scale

   \+ Simple to program, if synced correctly
   – Scalability limited by memory cost variation (even with NUMA) or bandwidth limiting speed
ii. Message Passing: highly structured
   - Tasks operate in private address spaces; communication via explicit send/receive
   - Matches distributed memory systems

   \+ Allows scaling across many machines
   – Complex programming
   – Network communication delays

Foster's Design Methodology

i. Partitioning: problem into pieces/data
   - $\geq 10\times$ primitive tasks than cores, of similar size
   - Minimize redundant computations and data
   - Number of tasks increases with problem size
ii. Communication: pass needed data among tasks
   - Balance communication among tasks
   - Perform communication in parallel
   - Overlap computation with communication
   - Communicate with small number of neighbours
iii. Agglomeration: combine tasks into larger tasks
   - Number of tasks $\geq$ number of cores
   - Reduce cost of task creation + communication
   - Maintain scalability, simplify code
iv. Mapping: tasks to execution units
   - Maximize processor utilization; minimize IPC
   - Optimal mapping is NP-hard $\Rightarrow$ heuristics
   - Consider static vs dynamic allocation; if static, tasks:cores $\geq$ 10:1; if dynamic, allocator must not bottleneck

Parallel Programming Patterns

Patterns:

i. Fork–Join: `fork` child tasks, then `join` (D&C)
ii. Parbegin–Parend: mark statements to execute in parallel threads, then sync at end
iii. SIMD: single instruction stream over multiple data elements (synchronous)
iv. SPMD: same program on multiple threads; different data via IDs (asynchronous)
v. Master–Worker: master distributes tasks to workers
vi. Task Pools: shared pool of tasks; fixed number of worker threads dynamically dequeue/insert tasks
vii. Producer–Consumer: produce tasks into buffer for consumers to retrieve; sync coordinates passing
viii. Pipelining: split compute into stages for data stream; throughput improves via stage-overlap

4. Performance

Performance is measured by primary goals:

i. Response Time: wall-clock duration of an execution consists of user, system and waiting time
ii. Throughput: average work completed per unit time

Sequential Programs

User CPU Time:

$$T_{user} = N_{cycle} \times T_{cycle}$$

i. $T_{cycle} = \frac{1}{\text{(average) clock freq.}}$
ii. $N_{cycle} = \sum n_i \times CPI_i \approx N_{instr} \times CPI$

User CPU Time (Memory Access; One-level Cache):

$$T_{user} = \big(N_{cycle} + N_{mm_cycle}\big) \times T_{cycle}$$

i. $N_{mm_cycle} = N_{read_cycle} + N_{write_cycle}$
ii. $N_{read_cycle} = N_{reads} \times R_{miss_rate} \times N_{miss_cycles}$

Average Memory Access Time (Two-level Cache):

$$T_{read} = T_{hit} + \big(R_{miss} \times T_{miss}\big)$$

i. $T_{read} = T_{hit}^{L1} + \big(R_{miss}^{L1} \times T_{miss}^{L1}\big)$
ii. $T_{miss}^{L1} = T_{hit}^{L2} + \big(R_{miss}^{L2} \times T_{miss}^{L2}\big)$
iii. Global miss rate: $R_{miss} = R_{miss}^{L1} \times R_{miss}^{L2}$

User CPU Time (Integrated CPI + Memory):

$$T_{user} = \Big(N_{cycle} + [N_{rw_ops} \times R_{miss} \times N_{miss_cycles}]\Big) \times T_{cycle}$$

Throughput Metrics:

i. MIPS $= \frac{N_{instr}}{T_{user} \times 10^6}$
ii. MFLOPS $= \frac{N_{fl_ops}}{T_{user} \times 10^6}$
   + Restrict to float ops (but variance exists)
iii. Arithmetic Intensity (AI) $= \frac{N_{fl_ops}}{N_{bytes}}$
   + Roofline: $P \leq \min\big(P_{\text{peak}},\ B \cdot \text{AI}\big)$ shows if bound by computation or communication bandwidth

Parallel Programs

Parallel Execution Time on p PUs, T_p consists:

i. Time for executing local computations
ii. Time for exchange of data between PUs
iii. Time for synchronization between PUs
iv. Waiting time

Speedup:

$$S_p = \frac{T_*}{T_p}$$

i. Measures the benefit of parallelism compared to best sequential algorithm time T_*
ii. Theoretically, $S_p \leq p$ always holds
iii. In practice, $S_p > p$ (superlinear speedup) can occur (e.g. problem working set "fits" in the cache)

Cost:

$$C_p = p \times T_p$$

i. Measures total work performed by all PUs i.e. processor-runtime product
ii. Cost-optimal: parallel program executes same total number of operations as fastest sequential program

Efficiency:

$$E_p = \frac{T_*}{C_p} = \frac{S_p}{p} = \frac{T_*}{p \times T_p}$$

i. Measures actual degree of speedup performance achieved compared to maximum
ii. Theoretically, ideal $S_p = p \ \Rightarrow\ E_p = 1$
iii. In practice, $E_p > 1$ can occur with superlinear speedups

Scalability

Scalability is the interaction between problem size and parallel machine size:

i. Impacts: load balancing, overhead, arithmetic intensity, locality (application dependent)
ii. Fixed problem size:
   - Small n: overheads dominate $\Rightarrow$ suitable for small machine, poor on large machine
   - Large n: working set may not fit on small machine (thrashing / exceeds cache / can't run)
iii. Constraints:
   - Application: particles/processor (N-body), transactions/processor (distributed DB)
   - Resources: time-limited, memory-limited, problem-limited (same problem faster)

Amdahl's Law:

$$S_p(n) = \frac{T_*(n)}{f \cdot T_*(n) + \frac{1-f}{p} T_*(n)} = \frac{1}{f + \frac{1-f}{p}} \leq \frac{1}{f}$$

i. Sequential fraction: $0 \leq f \leq 1$ limits speedup
ii. Discourage from making large parallel computers, focus on reducing sequential fraction f
iii. Rebuttal: circumvented for large problem size
   - Commonly f is a function of n, $f(n)$
   - Efficient parallel algorithm: $\lim_{n\to\infty} f(n) = 0$
   - Thus, $\lim_{n\to\infty} S_p = \frac{p}{1+(p-1)f(n)} = p$

Gustafson's Law:

$$S_p(n) = \frac{\tau_f + \tau_v(n,1)}{\tau_f + \tau_v(n,p)}$$

i. Encouraged to use more processors to solve larger problem in the same time (in CPU-bound apps)
ii. τ_f: constant sequential execution time
iii. $\tau_v(n,p)$: parallel execution time on p processors; if perfectly parallelizable (no overheads):
   - $\tau_v(n,1) = T^*(n) - \tau_f$
   - $\tau_v(n,p) = \frac{T^*(n) - \tau_f}{p}$
iv. If $T^*(n)$ increases strongly monotonically with n, then $\lim_{n\to\infty} S_p(n) = p$

OpenMP

Shared-Memory Programming Model:

i. Threads communicate via shared variables; may also have private variables
ii. Avoid race conditions using mutual exclusion

Constructs

General:

i. `#pragma omp parallel`: define parallel region (fork–join), creating team of threads in the block
ii. `omp_get_thread_num()`/`omp_get_num_threads()`: query a thread's id and the current team size.
iii. `omp_set_num_threads(n)` / `OMP_NUM_THREADS`: control the number of threads
iv. `nowait`: disable implicit barrier after blocks

Data Parallel For:

i. `#pragma omp for`: work-sharing for loops
ii. `schedule(kind,chunksize)`: control static vs. dynamic allocation and chunking
iii. `shared(...)`/`private(...)`: data-scoping clauses

Controlled:

i. `#pragma omp sections`/`#pragma omp section`: work-sharing over per-thread sections
ii. `#pragma omp single`: one thread executes block
iii. `#pragma omp master`: only master executes block

Synchronisation:

i. `#pragma omp barrier`: explicit barrier for threads
ii. `#pragma omp critical`: critical section block
iii. `#pragma omp atomic`: atomic update to a location

Mutex:

```
omp_lock_t lock;
omp_init_lock(&lock);
#pragma omp parallel
{
  omp_set_lock(&lock);
  x++;
  omp_unset_lock(&lock);
}
```

Appendix

name	prefix	multiplier
exa	E	10^{18}
peta	P	10^{15}
tera	T	10^{12}
giga	G	10^{9}
mega	M	10^{6}
kilo	K	10^{3}
milli	m	10^{-3}
micro	μ	10^{-6}
nano	n	10^{-9}
pico	p	10^{-12}

name	prefix	multiplier
exbi	Ei	2^{60}
pebi	Pi	2^{50}
tebi	Ti	2^{40}
gibi	Gi	2^{30}
mebi	Mi	2^{20}
kibi	Ki	2^{10}